11. GPT 3

17 March 2026 11:27

until this → pretraining on large datasets followed by fine tuning
↓ learns general linguistic representations
↓ modifies parameters for task-specific decision boundaries

GPT-2 demonstrated that zero-shot behaviour emerges from large language models

GPT-1 pretrain → fine-tune
GPT-2 pretraining alone gives zero shot behavior
(GPT-3) In context learning (few shot) emerges when scaling
↓
task definition occurs inside the prompt

paper tests few shot ability being a scaling property
↳ more params + more data + more compute → stronger in context learning

Objective remains same → predict next token

Representation → model learns a distn over text sequences
→ task behavior emerges from here

Information flow → Task instructions + examples are embedded as next tokens

Optimization → happens during pretraining only
Evaluation = pure inference

Can a language model perform a new task without parameter updates, simply from examples in context?

→ In-context learning
few shot learning

few shot exploits the behaviour of model recognising repeated pattern structures in text

few shot prompting works because
Language modeling ≈ pattern completion

Scaling allows models to learn algorithms inside the network

small models → only learn surface correlations
large models → structured transformations

Pattern Extraction, Task inference, Pattern Application through few shot application

Few shot learning ability emerges through as a scaling property

Introduction Removing the fine tuning on dataset of thousands to hundreds or needing of task specific datasets is desirable

Introduction Removing the fine tuning on dataset of thousands to hundreds or needing of task specific datasets is desirable

→ need for large datasets of labeled examples → limits the applicability of language models

Potential to exploit spurious correlations in training data fundamentally grows with expressiveness of the model and narrowness of training distribution

Models were designed large to absorb information and then fine tuned on narrow distributions

larger model do not necessarily generalize better out of distribution

humans do not need large supervised datasets to learn most language tasks

Large models were increasingly efficient use of incident information

Meta learning → model develops a broad set of skills and pattern recognition abilities at training time & uses those abilities at inference to rapidly adapt or recognize the desired task

They say fine tuning causes distribution narrowing (can cause overfitting to task format)

There was label dependence (Many language tasks do not have large labeled datasets)

Traditional systems have task specification rigidly

They proposed task = natural language prompt

Allows task specification through text

outer loop = pretraining

inner loop = task learning from prompt

from plots → few shots improve fastest with scale → large models benefit more with demonstrations

Gap (few shot, zero shot) increases with scale

Approach -

basic pretraining approach → similar to GPT-2

fine tuning [FT] - involves updating wts of a pretrained model by training on a supervised dataset specific to desired task

→ Advantage → strong performance on many benchmarks

disadvantage → need for a new large dataset for every task

poor generalization out of distrib^n

potential to exploit spurious features of training data

They dont do it as task agnostic performance is focus

Few shot (FS) Model is given few demonstrations of task at inference time as conditioning, but no wt updates are allowed

adv red^n in need for task specific data

reduced potential to learn an overly narrowed distr^n

disadv results worse than SOTA

Small amount of task specific data still required

One Shot (1S) only one demonstration allowed in addition to natural language description of the task

Zero Shot (0S) No demonstrations allowed

Model and Architectures

GPT-2 including modified initialization, pre normalization and reversible tokenization → in BPE

↳ The exception being

Using alternate dense and locally banded sparse attention patterns in the layers of the transformer

↓

Similar to sparse transformer

With enough training data Scaling of validation loss should be approximately a smooth power law as a function of size

n_{params} — total no of params

n_{layers} → total no of layers

d_{model} → the no of units in each bottleneck layer

(we always have FFN to be 4 times bottle neck layer $d_{ff} = 4 * d_{model}$)

d_{head} → dimension of each attention head

n_{ctx} = 2048 tokens (context window)

partition the model across GPUs along both the depth and width dimension in order to minimize data transfer btw nodes

The precise no features for all tested model varieties are chosen based computational efficiency and load balancing in layout of models across GPU's

few/zero/one-shot = no weight updates → Only forward pass

Spurious correlations → Models exploit dataset shortcuts Instead of learning the real task

Suppose in training data eg positive reviews contain 'i'

Model might learn 'i' positive sentiment

Dense attention (normal transformer)

each token attends to all tokens

$O(n^2)$

Sparse attention

Each token attends to only subset of tokens

local window + periodic global tokens

Model parallelism across layers

GPU1 → layers 1-12

GPU2 → layers 13-24

} pipeline parallelism

Model parallelism within Matrices

$W = [W_1 | W_2]$

Each GPU stores part

The results are combined

} Tensor parallelism

Higher quality datasets sampled more frequently.

The results are combined

Higher quality datasets sampled more frequently

Fuzzy deduplication detects near-duplicates

Larger model → large batch size → smaller learning rate

→ few shot evaluation, likelihood for multiple choice tasks, Beam search for generation tasks

Power law $loss = a * N^{(-b)}$

↳ model size / compute / data

→ language modeling/completion few shot > zero shot

↳ examples help model infer the task format

→ Closed Book Q/A

Model must answer from knowledge stored in parameters (no retrieval)

few shot GPT 3 sometimes matches retrieval based systems

But GPT-3 stores knowledge in wts

RAG retrieves it from database

→ Translation → Not trained for this

But few shot still works well {training data mostly in english}

translation → English > translation from English

Pronoun reasoning (winograd) → near SOTA performance

Commonsense reasoning → Good but below human level

Reading comprehension → fine tuned models > GPT3

few shot doesn't yet replace fine-tuning

*NLU → GPT3 struggles

Synthetic tasks like arithmetic, novel word usage, word manipulation

{performance increases dramatically with model scale}

* Model size↑

few shot performance ↑↑ } emergent meta learning behavior

Data contamination → training data (internet) contains benchmark test examples

Model then memorizes answers

→ They searched for overlaps, removed some contaminated tests, analyzed impact

• Only then memorizes answer

→ They searched for overlaps · removed some contaminated tasks, analyzed impact

Contamination exists but does not explain most results
(Real risk)

Limitations

i) Hallucination / Inconsistency → models contradict themselves, repeat text, produce nonsense

ii) Reasoning weakness

iii) Pretraining Objective limits → maximize P(next token | context)

real tasks require reasoning, planning, fact verification, goal optimization

They are not explicitly optimized

iv) Sample inefficiency

GPT3 training data - 300B tokens

humans learn language with ~ 50M tokens

LLM's require huge data

v) few shot ambiguity

Does the model learn new tasks?

or

Does it recognize tasks it saw during training?

vi) Massive Compute costs

vii) Lack of interpretability

Distillation → train small model to imitate a big model

procedure → Teacher model (large)
↳ generates output
↳ student model learns from those outputs

Goal → retain ability with smaller model

Quantization → reduce precision of weights

FP32 → FP16 → INT8 - INT4

reduce memory → speed up inference

Related work

Pretrained language models → large pretrained models → better downstream performance

Meta learning / few shot learning → train models to adapt to new tasks quickly

Meta learning / few shot learning → train models to adapt to new tasks quickly

Scaling laws → loss follows power law with s → GPT-3 extends this

Language models are general task solvers

task = text prompt

Instead of task = dataset + finetuning

* Scaling enables in-context learning
* few shot performances grow with scale
* No gradient updates needed at inference
* Broad task coverage
* Important limitations remain → hallucinations
  poor reasoning
  compute costs
  data contamination